Assignment 7.2 - Identifying Bottlenecks and Proposing System-Wide Improvements

Date: September 23, 2026
Course: WEB430
Professor: Richard Krasso
Student: Jonathan Cantu

Part I: HorizonFleet Logistics Workflow Analysis

HorizonFleet Logistics is experiencing several problems that appear separate on the surface but are likely interconnected. Long code reviews, unreliable tests, manual deployment steps, communication gaps, and unresolved incidents all interrupt the movement of work through the development pipeline. The first priority should therefore not be adding more tools. It should be identifying where work consistently waits, fails, or requires rework and then improving the system around those constraints.

1. Identify Bottlenecks

Code review appears to be one of HorizonFleet's most visible bottlenecks. If pull requests remain in review for long periods, completed development work simply moves into a queue instead of progressing toward deployment. Krasso (2026) explains that accumulated work, repeated waiting on individuals or groups, and delays near production are common signs of system constraints.

Frequent test failures create a second constraint, particularly because the failures are difficult to diagnose. Tests are supposed to provide fast Feedback, but unreliable or poorly understood failures can have the opposite effect. Developers may spend significant time rerunning pipelines, searching logs, or determining whether the problem is actually related to their change.

Manual deployment steps add additional waiting and variability. At the same time, inconsistent communication between development and operations creates handoff problems, while the backlog of unresolved incidents indicates that lessons from operational failures are not consistently making their way back into the development workflow.

These problems should be viewed as a system rather than as isolated weaknesses. As Krasso (2026) notes, software delivery moves at the speed of its constraint, regardless of how efficient individual portions of the workflow may be.

2. Evaluate System-Wide Impact

These bottlenecks directly weaken Flow because work repeatedly stops between development, review, testing, deployment, and operations. Improving developer coding speed alone would not solve the problem. In fact, producing code faster while review remains constrained could simply increase the backlog of waiting pull requests.

Feedback also becomes weaker when tests fail unpredictably or pipeline failures are difficult to understand. A signal only has value when the team trusts it and can act on it quickly. Likewise, unresolved incidents represent lost Feedback because the organization is seeing production problems without consistently converting those events into corrective actions.

The overall result is unpredictability. Developers cannot confidently estimate how long a change will take to reach production because the time spent waiting, troubleshooting, or coordinating varies from one change to another.

3. Recommend System-Wide Improvements

I would begin by measuring where work is actually waiting. HorizonFleet should track pull request review time, pipeline duration, test failure frequency, deployment lead time, and unresolved incident age. Krasso (2026) emphasizes that data such as cycle time, lead time, build failures, incidents, and rework should be used to understand the system rather than evaluate individual developers.

GitHub Actions could then be used to automate repeatable quality checks earlier in the workflow. A pull request workflow could run formatting, linting, unit tests, and build validation before a human reviewer begins detailed review. GitHub supports required status checks that can prevent a pull request from merging until defined validations have passed (GitHub, 2026).

Prek could provide another early quality gate. It is designed as a Git hook manager and can also run through GitHub Actions, allowing the same checks developers run locally to execute automatically against pull requests. This could help prevent reviewers from spending time on mechanical issues that automation can identify first.

Security testing should also be placed thoughtfully. Invicti supports integration with GitHub Actions and CI/CD workflows, making automated application-security scanning possible (Invicti, 2026). However, I would not automatically run a lengthy security scan on every small code change if doing so creates another pipeline constraint. Lightweight checks could run on each pull request, while deeper dynamic scans could execute after deployment to a staging environment or at another appropriate quality gate.

Deployment steps that are currently manual should also be evaluated for automation. If the same repeatable steps occur every time, GitHub Actions could provide a consistent deployment workflow while retaining human approval where the level of risk justifies it.

Finally, the incident backlog needs ownership. Regular incident review should identify recurring causes, failed controls, or missing tests and feed those findings back into development.

4. Propose a Small Experiment

Rather than redesigning the entire workflow, I would begin with a two-week experiment on one repository.

The hypothesis would be: “If HorizonFleet automates routine code-quality checks before human review, then average pull request review time will decrease without increasing escaped defects.”

A GitHub Actions workflow could run Prek checks, unit tests, and the application build whenever a pull request is opened or updated. Those jobs could become required status checks before merge.

Before beginning the experiment, the team should establish a baseline for average review time, first-review rejection rate, pipeline failures, and defects discovered after merge. The same measurements could then be collected during the pilot.

If reviews become faster without sacrificing quality, the workflow could be expanded gradually. If the automation instead increases waiting or produces excessive noise, the team should adjust it rather than assume more automation is automatically better. This follows Krasso's (2026) hypothesis-driven cycle of identifying a problem, making a small change, measuring the result, and adjusting based on evidence.

5. Explain How AI Could Support the Process

AI-assisted tools could help HorizonFleet analyze the information already being produced by the workflow. For example, AI could summarize failed GitHub Actions logs, group recurring failures, identify flaky tests, or analyze incident history to reveal patterns that are difficult to see manually.

This could be particularly helpful with the unresolved incident backlog. An AI tool could classify incidents by common symptoms, affected components, or likely causes and help the team identify recurring system weaknesses.

However, AI should support diagnosis rather than automatically determine the solution. Krasso (2026) emphasizes that AI can find patterns and identify bottlenecks but cannot fully understand organizational priorities, trade-offs, or long-term consequences. Engineers should validate the AI's conclusions before changing the workflow.

6. Connect to the Three Ways

These recommendations support all Three Ways:

- **Flow** improves by reducing waiting during code review, automating repeatable checks, removing unnecessary deployment steps, and preventing avoidable problems from reaching later stages.
- **Feedback** improves when automated quality checks, reliable tests, security scanning, and clearer pipeline diagnostics identify problems earlier and provide developers with actionable information.
- **Continuous Learning** improves when HorizonFleet measures workflow performance, reviews incidents, evaluates experiments, and adjusts processes based on evidence. AI can accelerate that learning by identifying trends across build failures, test results, and incident data, but the team remains responsible for interpreting and acting on those insights.

The larger goal is not to automate every possible step. It is to create a workflow in which work moves predictably, Feedback arrives early, and the organization continually learns from measurable results rather than assumptions.

References:

GitHub. (2026). "Status checks". GitHub Docs. https://docs.github.com/en/pull-requests/reference/status-checks

Invicti. (2026). "Automate security workflows". Invicti. https://www.invicti.com/solutions/automate-security-workflows

Krasso, R. (2026). "WEB 430 Weekly Reading: Week 7". Bellevue University.

Kulkarni, T. (2026, March 30). "Automating code quality checks with GitHub Actions". NamasteDev. https://www.invicti.com/solutions/automate-security-workflows

Part II: Evaluation of AI Assisted Analysis of Incident Log

For this part of the assignment, you will work with a simulated incident log and use an AI-assisted tool to:

- Summarize the incident
- Identify patterns or root causes
- Suggest improvements
- Evaluate the quality of the AI's analysis

This gives you practical experience using AI to support problem-solving and continuous improvement.

Incident Log (TXT File):

```
2024-10-12 09:14:23 ERROR: Build failed during test phase. Timeout exceeded.
2024-10-12 09:15:10 INFO: Retrying test suite.
2024-10-12 09:16:42 ERROR: Test suite failed. Module: route-optimizer.
2024-10-12 09:18:05 WARN: Flaky test detected in route-optimizer.
2024-10-12 09:20:11 ERROR: Deployment to staging delayed due to unresolved test
failures.
2024-10-12 09:45:33 INFO: Manual override triggered by operations.
2024-10-12 10:02:19 ERROR: Incident reopened. Root cause not identified.
```

Steps:

1. Copy the incident log into an AI-assisted tool.
2. Ask the tool to summarize the incident.
3. Ask the tool to identify patterns or root causes.
4. Ask the tool to suggest improvements.
5. Review the output and reflect on what you notice.

I used Deepseek as the AI Tool.

Deepseek Response

📋 Incident Summary

A series of test failures and errors occurred during a deployment process, ultimately delaying staging deployment and reopening an incident. Key events include:

- **Initial Build Failure** (09:14:23): Timeout during test phase.
- **Retry & Module Failure** (09:16:42): Test suite failed specifically in the `route-optimizer` module.
- **Flaky Test Detection** (09:18:05): Intermittent test issue identified in the same module.
- **Deployment Delay** (09:20:11): Staging deployment paused due to unresolved test failures.
- **Manual Intervention** (09:45:33): Operations team triggered a manual override.
- **Incident Reopened** (10:02:19): Root cause remained unidentified, leading to the incident being reopened.

🔍 Patterns & Root Causes

- **Recurring Module Issue**: Multiple errors (`ERROR` and `WARN`) center around the `route-optimizer` module.
- **Timeout & Flakiness**: Initial failure due to timeout, followed by detection of a flaky test, suggesting unstable or resource-intensive tests.
- **Lack of Root Cause Resolution**: Despite retries and manual override, the underlying issue wasn't resolved, causing the incident to reopen.
- **Manual Override Dependency**: Reliance on manual intervention indicates gaps in automated recovery or failure handling.

💡 Suggested Improvements

- **Immediate Actions**:
  - Isolate and fix the flaky test in `route-optimizer` to prevent recurring failures.
  - Increase timeout thresholds or optimize test performance for resource-heavy modules.
- **Process Enhancements**:
  - Implement automated rollback or fallback mechanisms to avoid manual overrides.
  - Introduce better logging and monitoring to capture root causes during failures.
- **Long-Term Strategies**:
  - Strengthen test suite reliability through regular flaky test detection and elimination.
  - Establish a post-incident review process to document and address unresolved root causes.
  - Enhance deployment pipelines with better failure recovery and alerting to reduce downtime.

AI-Generated Incident Summary

DeepSeek summarized the incident as a failed build during the test phase that eventually delayed deployment to staging. The first failure was caused by a test timeout, followed by a retry that failed in the route-optimizer module. A flaky test was then detected in that same module, and the unresolved failures prevented staging deployment from proceeding normally. Operations later used a manual override, but the incident was reopened because the underlying cause had not been identified.

AI-Generated Root Cause Analysis

DeepSeek identified several patterns in the incident:

- Multiple failures were associated with the route-optimizer module.
- A timeout occurred during testing.
- A flaky test was detected shortly afterward.
- The deployment process depended on manual intervention when automated checks failed.
- The team proceeded with an override before determining the actual cause of the test failures.
- The incident was reopened because the problem had not been fully resolved.

DeepSeek suggested that unstable or resource-intensive tests in route-optimizer were a likely cause. However, the log only establishes correlation. It does not provide enough information to prove that the flaky test caused the original timeout or subsequent failures.

AI-Generated Improvement Suggestions

DeepSeek recommended:

- Isolating and correcting the flaky test in route-optimizer.
- Investigating test performance and determining whether the timeout reflects a slow test or another resource problem.
- Improving logging and monitoring so future failures provide more diagnostic information.
- Reducing dependence on manual overrides.
- Establishing a formal post-incident review process.
- Tracking and eliminating flaky tests before they become accepted pipeline behavior.
- Improving pipeline failure handling and alerting.

DeepSeek also suggested increasing timeout thresholds and implementing automated rollback or fallback mechanisms. Those recommendations would require additional investigation before implementation because increasing a timeout could hide a performance problem, and a rollback may not apply when deployment to staging never actually occurred.

Reflection

DeepSeek correctly identified the concentration of failures around the route-optimizer module and recognized the flaky test as an important signal that should be investigated. It also correctly connected unreliable testing to the staging delay and recognized that the manual override did not resolve the underlying problem. I particularly agree with its recommendation to establish a post-incident review process because the reopened incident shows that the team moved forward without understanding why the failure occurred.

One thing DeepSeek did not clearly distinguish, however, is the difference between correlation and root cause. The log shows a timeout, a failure in route-optimizer, and a flaky-test warning, but it does not prove that the flaky test caused the original timeout. I would also be cautious about simply increasing the test timeout because that could hide a performance problem instead of correcting it. Its rollback recommendation seems less applicable because the log indicates that deployment to staging was delayed rather than that a failed deployment needed to be reversed.

Overall, the suggestions around better logging, flaky-test remediation, failure investigation, and incident review are practical and support a more reliable pipeline. This connects directly to continuous improvement because the team should use the incident as data, form a hypothesis about the actual constraint, make a targeted change, and measure whether the same failure pattern occurs again.

Reference:

Krasso, Richard (2026). “WEB 430 Weekly Reading: Week 7”. Bellevue University.